Team Tasks - Group Submission

Final Project Report

Group 8

Chirag Rudresh, Justin Dam, Lwin Moe

Prof. Suneuy Kim

SP26: CS-157C Sec 81 - NoSQL

Department of Computer Science, San Jose State University

Apr 25, 2026

Team 8 Information:

- Chirag Rudresh (chirag.rudresh@sjsu.edu)
- Justin Dam (justin.dam@sjsu.edu)
- Lwin Moe (lwin.moe@sjsu.edu)

Property Graph Schema

Graph model:

The system uses a property graph model implemented in Neo4j.

Node type: (:User)

Property	Type	Description
user_id	String	Unique identifier
username	String	Unique username
name	String	Full name
email	String	Unique email
password	String	User password
bio	String	Profile description
country	String	Country
age	Integer	Age
signup_date	String	Registration date
activity_score	Float	Synthetic engagement metric

Relationship Type:

(:User)-[:FOLLOWS]->(:User)

Represents a direct follow relationship. Ex: If user A follows user B then

(A)-[:FOLLOWS]->(B)

Dataset Information

The dataset consists of multiple ego-networks collected from Facebook, where each file represents a user's network of friends. Each file contains undirected edges representing friendships between users.

Name: SNAP Facebook Ego Networks

URL: https://snap.stanford.edu/data/ego-Facebook.html

Preprocessing

The raw dataset consisted of multiple .edges files representing ego networks. Each file contained undirected edges between users.

The preprocessing pipeline included the following steps:

1. All ego-network files were merged into a single global graph.
2. Duplicate edges were removed and self-loops were eliminated.
3. A unified list of unique users was extracted.
4. Since the dataset did not include user profile information, synthetic attributes such as username, email, password, bio, country, age, and activity score were generated.

5. Each undirected edge (A, B) was converted into two directed edges (A → B and B → A) to represent the "FOLLOWS" relationship required for the application.

6. The final dataset was validated to ensure uniqueness of users, absence of duplicate relationships, and correctness of references.

The processed data was exported into two CSV files:

- users_final.csv
- follows_final.csv

Cypher statements used to create data

Users:

```
LOAD CSV WITH HEADERS FROM 'file:///users_final.csv' AS row
CREATE (:User {
user_id: row.user_id,
username: row.username,
name: row.name,
email: row.email,
password: row.password,
bio: row.bio,
country: row.country,
age: toInteger(row.age),
signup_date: row.signup_date,
activity_score: toFloat(replace(row.activity_score, ",", ""))
});
```

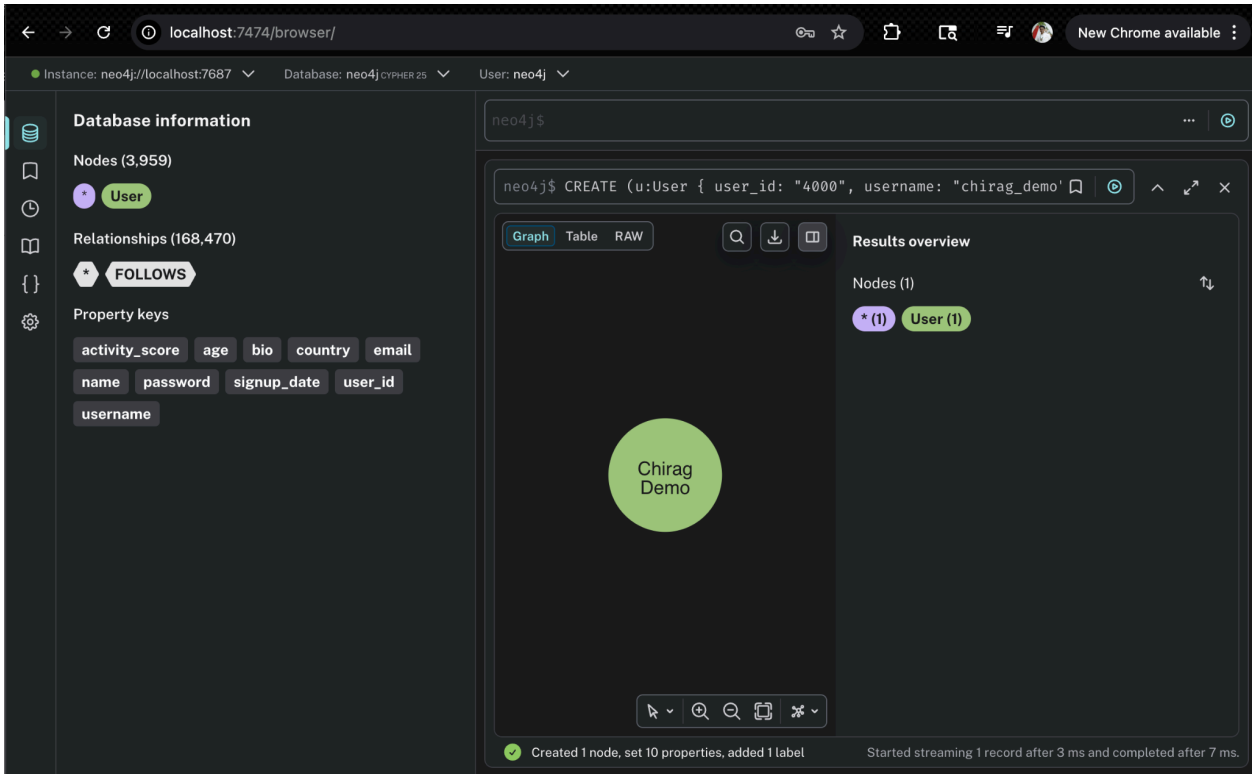
Follows:

```
LOAD CSV WITH HEADERS FROM 'file:///follows_final.csv' AS row
MATCH (a:User {user_id: row.follower_id})
MATCH (b:User {user_id: row.followee_id})
CREATE (a)-[:FOLLOWS]->(b);
```

Use Case Evidence

1) **UC-1: User Registration:**

```
CREATE (u:User {
  user_id: "4044",
  username: "chirag_demo",
  name: "Chirag Demo",
  email: "chirag.demo@example.com",
  password: "demo123",
  bio: "New user account created through registration.",
  country: "US",
  age: 25,
  signup_date: toString(date()),
  activity_score: 0.5
})
RETURN u;
```



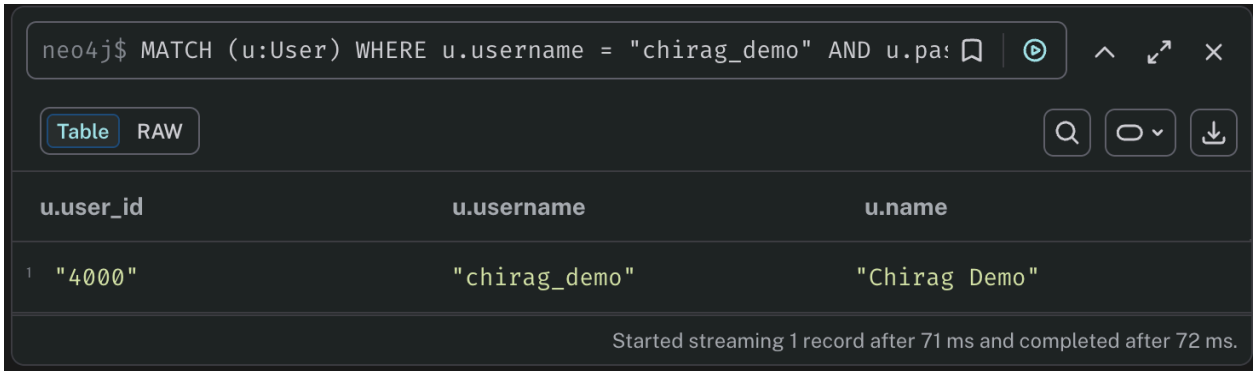
The user is registered successfully in the database through the create account feature.

2) **UC-2: User login**

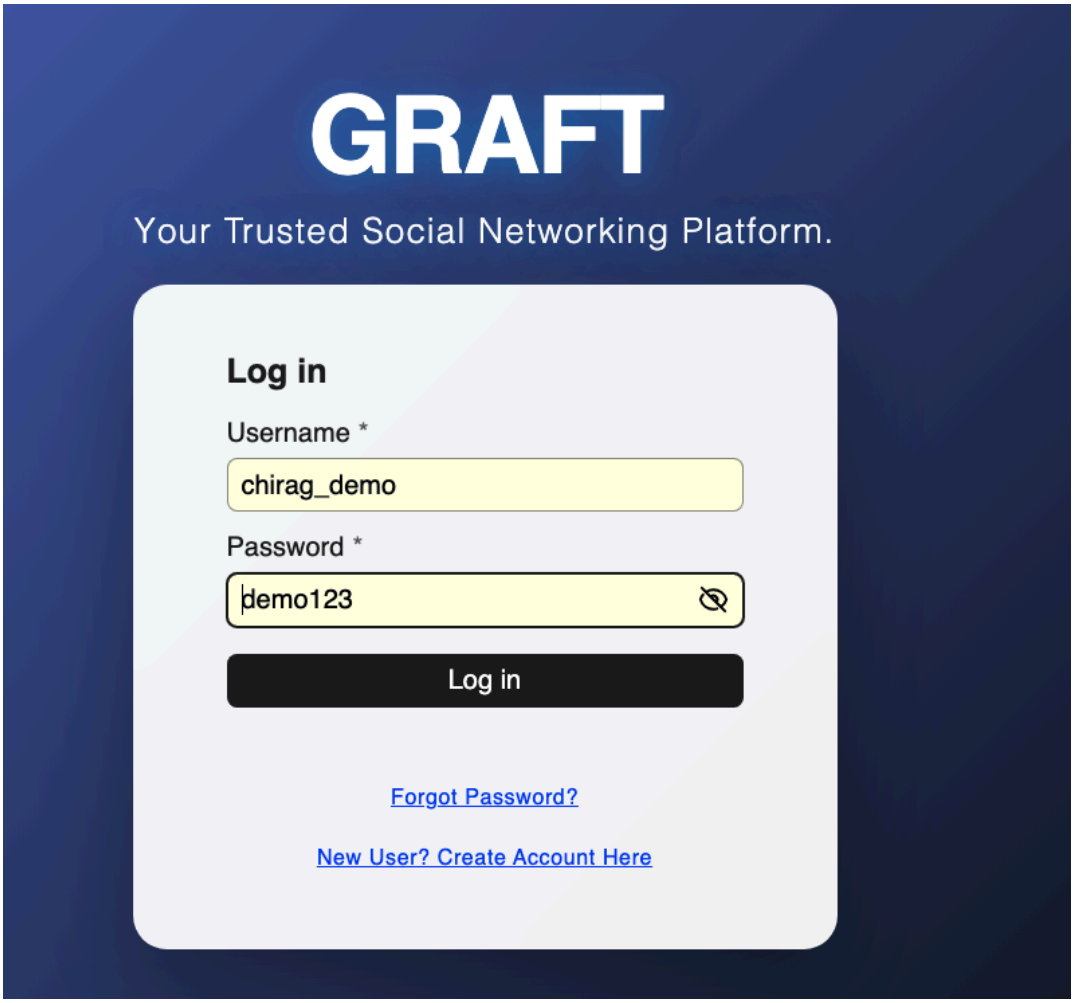
MATCH (u:User)

WHERE u.username = "chirag_demo" AND u.password = $hash_password

RETURN u.user_id, u.username, u.name;



The user was able to log in using the password he created during account setup.



3) UC-3: View Profile

MATCH (u:User {user_id: 4044})

RETURN u.user_id AS user_id,

u.username AS username,

u.name AS name,

u.email AS email,

u.bio AS bio,

u.country AS country,

u.age AS age,

u.signup_date AS signup_date,

u.activity_score AS activity_score;

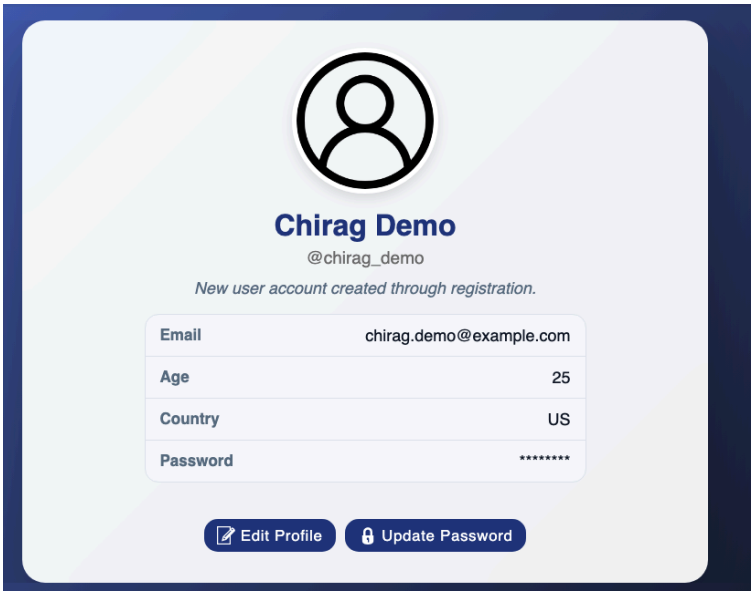
```
MATCH (u:User {user_id: 4044})
RETURN u.user_id AS user_id,
       u.username AS username,
       u.name AS name,
       u.email AS email,
       u.bio AS bio,
       u.country AS country,
       u.age AS age,
       u.signup_date AS signup_date,
       u.activity_score AS activity_score;
```

Table RAW

user_id	username	name	email	bio	country	age
4044	"chirag_demo"	"Chirag Demo"	"chirag.demo@example.com"	"Updated bio for the social network project demo."	"US"	25

Started streaming 1 record after 28 ms

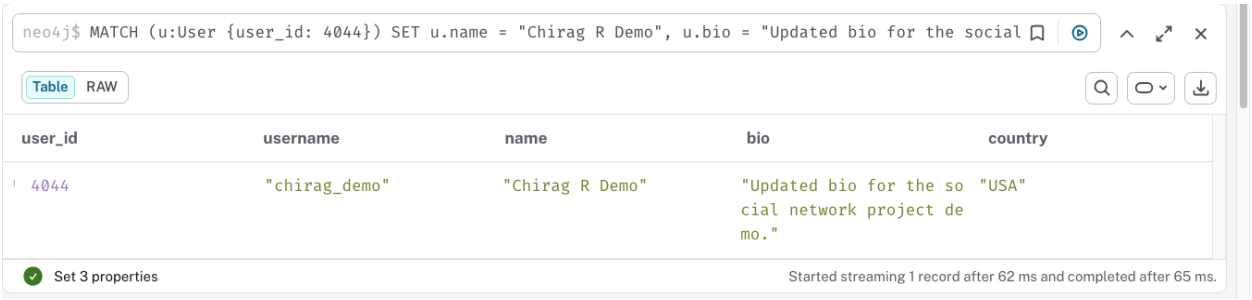
The provided property key is not in the database

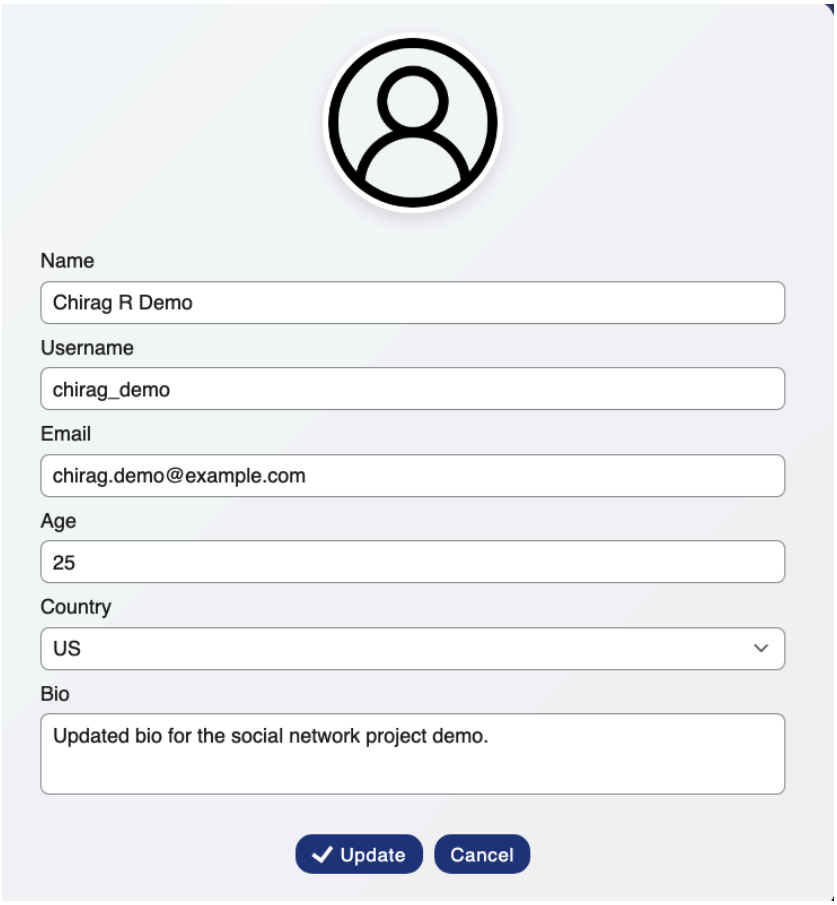


The user is able to view his profile on the website.

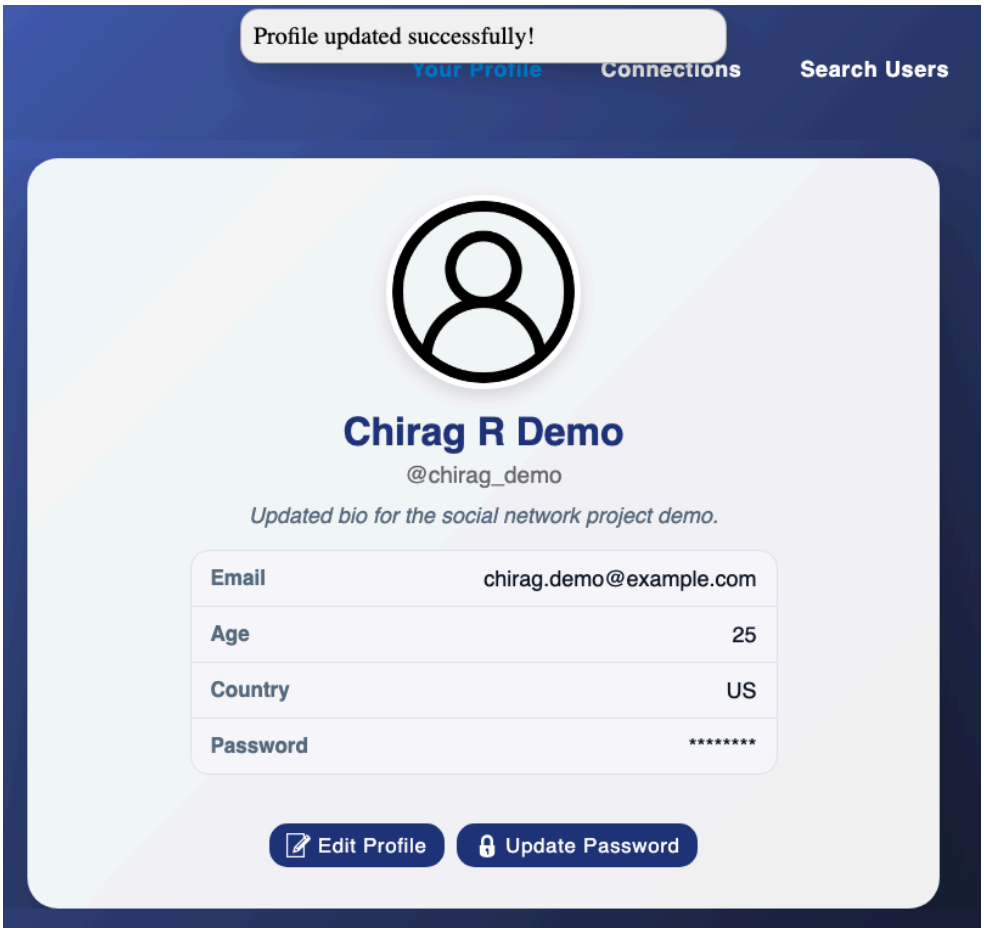
UC-4: Edit Profile

```
MATCH (u:User {user_id: 4044})
SET u.name = "Chirag R Demo",
    u.bio = "Updated bio for the social network project demo.",
    u.country = "USA"
RETURN u.user_id AS user_id,
       u.username AS username,
       u.name AS name,
       u.bio AS bio,
       u.country AS country;
```





The user updated his bio through the edit profile feature on the website. The second photo and database query result shows that the user was able to update his bio successfully.



Social Graph Features

New user A(id:4000) and old user B(10) taken for consistency purposes.

4) **UC-5: Follow Another User**

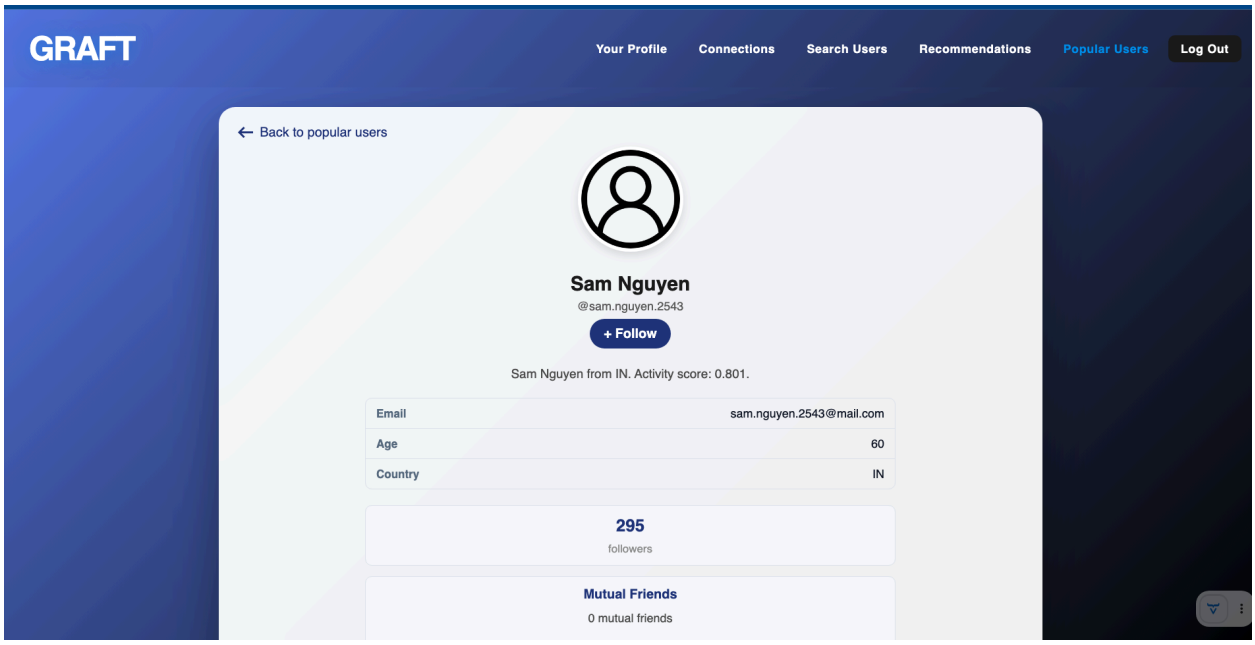
MATCH (a:User {user_id: 4044})

MATCH (b:User {user_id: 2543})

MERGE (a)-[:FOLLOWS]->(b)

RETURN a.user_id AS follower, b.user_id AS followee;

Before running the query, User 2543 (Sam Nguyen) is not followed by Chirag (User 4044).




```
1 MATCH (a:User {user_id: 4044})
2 MATCH (b:User {user_id: 2543})
3 MERGE (a)-[:FOLLOWS]->(b)
4 RETURN a.user_id AS follower, b.user_id AS followee;
5
```

Table RAW

follower	followee
4044	2543

Created 1 relationship

Started streaming 1 record after 32 ms and completed after 36 ms.

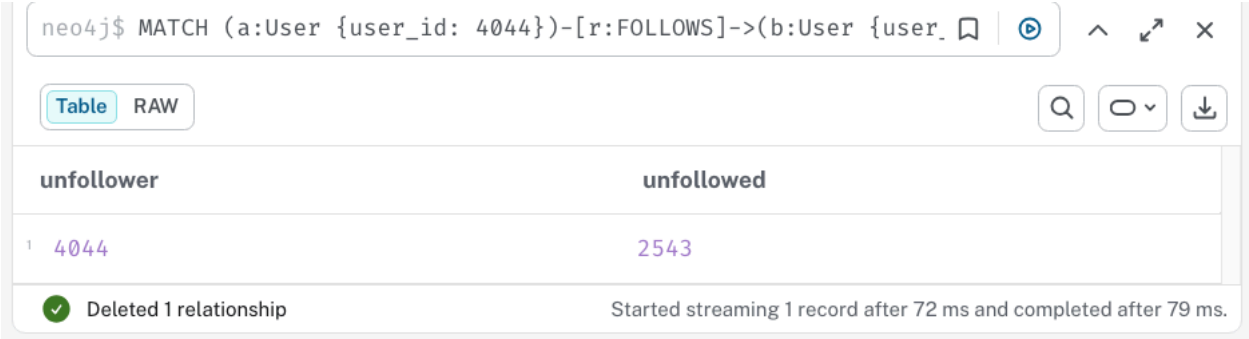
After running the query, there is a "follows" relationship between User 2543 (Sam Nguyen) and User 4044 (Chirag).

5) UC-6: Unfollow a User

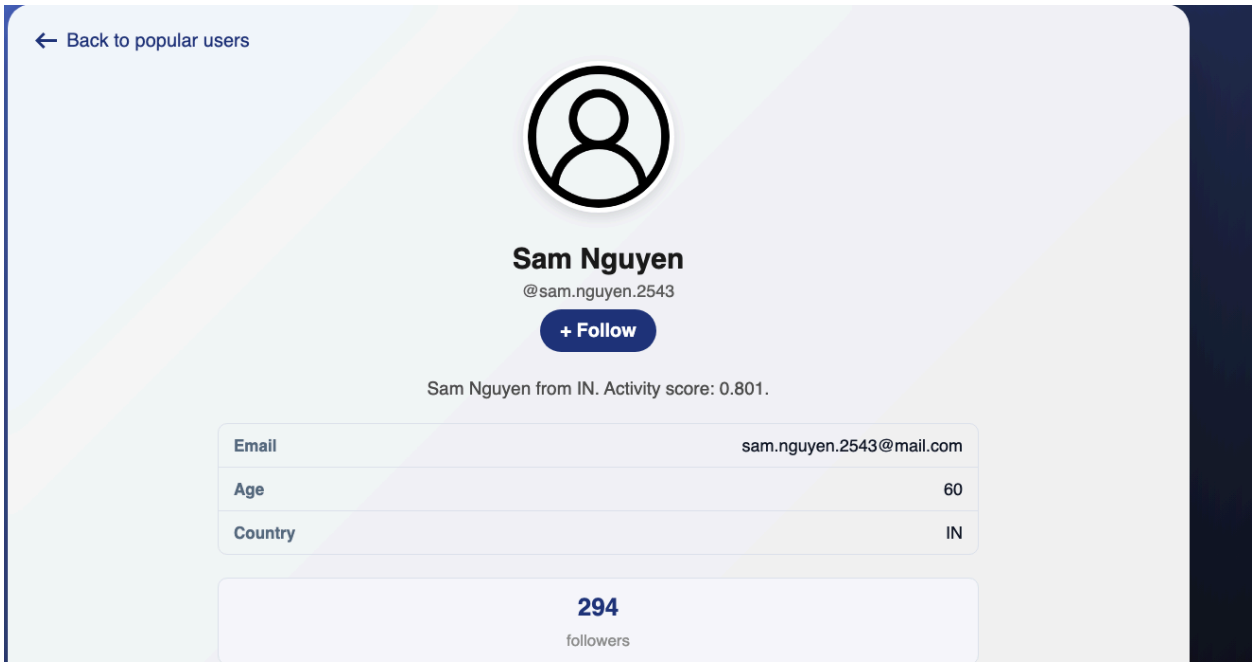
MATCH (a:User {user_id: 4044})-[r:FOLLOWS]->(b:User {user_id: 2543})

DELETE r

RETURN a.user_id AS unfollower, b.user_id AS unfollowed;



After running the query, there is a button to follow User 2543 (Sam) since there is no relationship between Chirag (User 4044) and User 2543 in the database. The relationship was deleted.



The above follow/unfollow operations can be done on the website as well. The features are working as intended.

6) UC-7: View Friends/Connections:

Check following:

MATCH (u:User {user_id: 4044})-[:FOLLOWS]->(v:User)

RETURN v.user_id AS user_id, v.username AS username, v.name AS name

LIMIT 10;

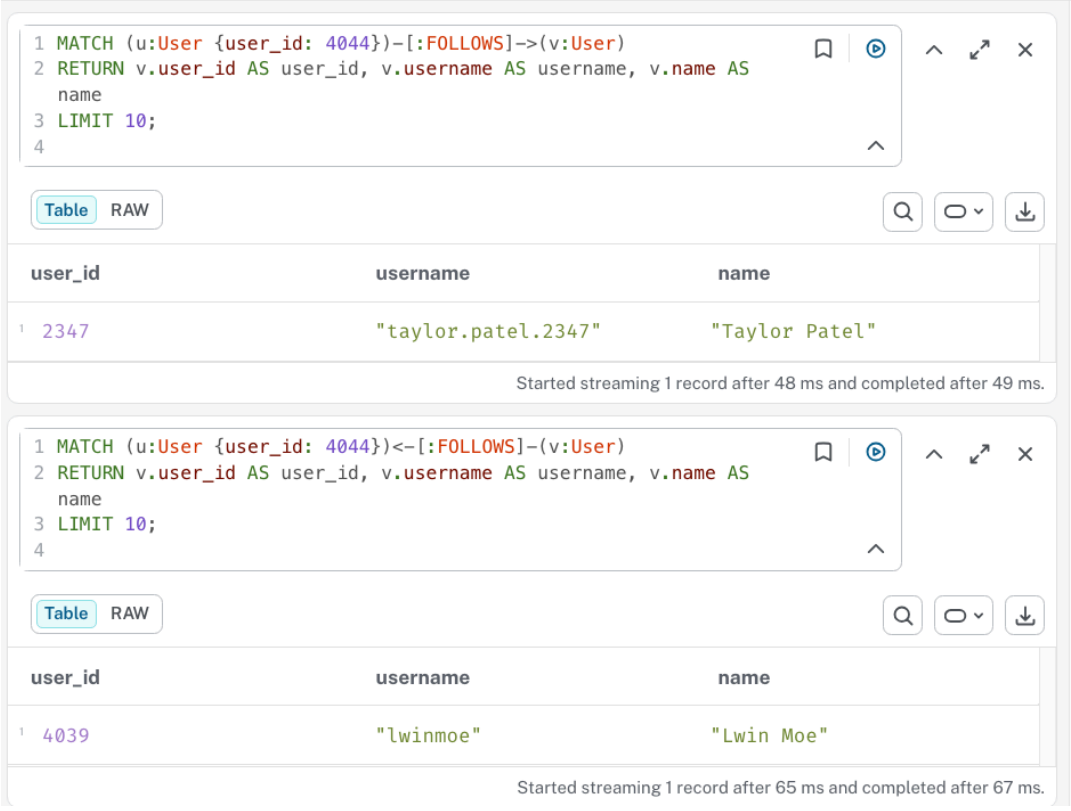
Check followers:

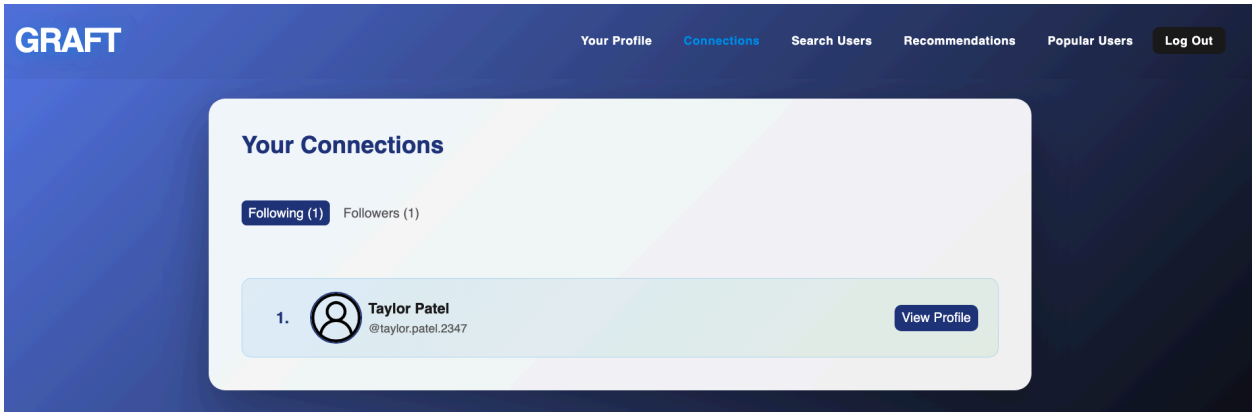
MATCH (u:User {user_id: 4044})<-[:FOLLOWS]-(v:User)

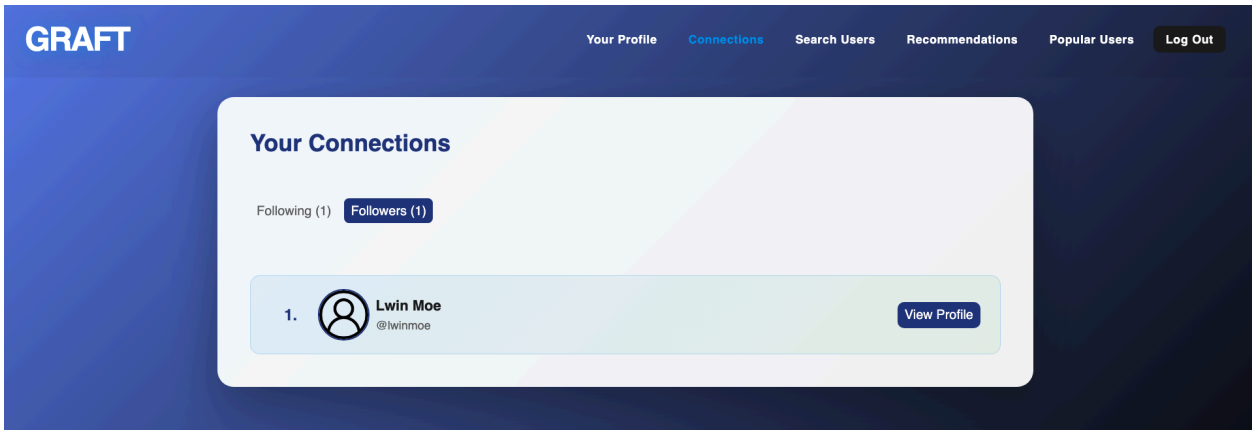
RETURN v.user_id AS user_id, v.username AS username, v.name AS name

LIMIT 10;

- Shows who the user follows and who follows the user







As you can see, the query results reflect what is showing on the website. You can see who you follow and who your followers are.

7) **UC-8: Mutual Connections**

MATCH (a:User {user_id:

4044})-[:FOLLOWS]->(m:User)<-[:FOLLOWS]-(b:User {user_id: 4039})

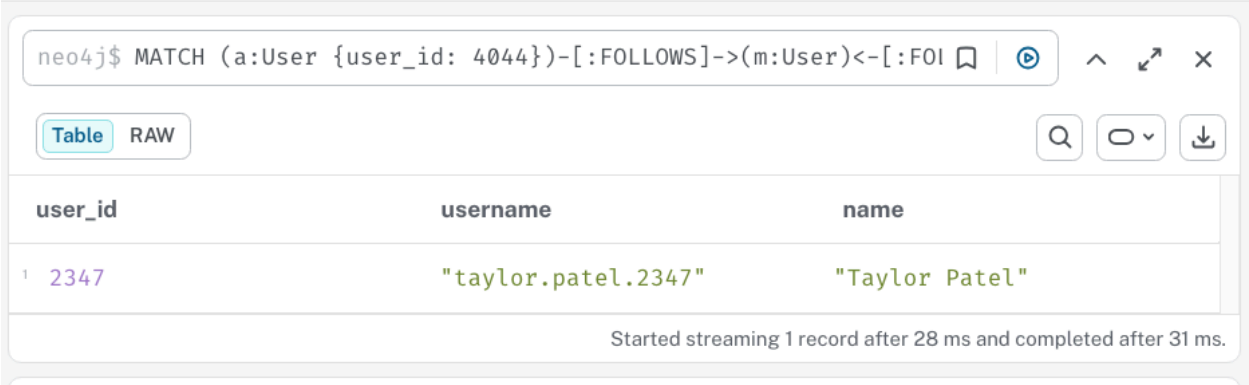
RETURN m.user_id AS user_id,

m.username AS username,

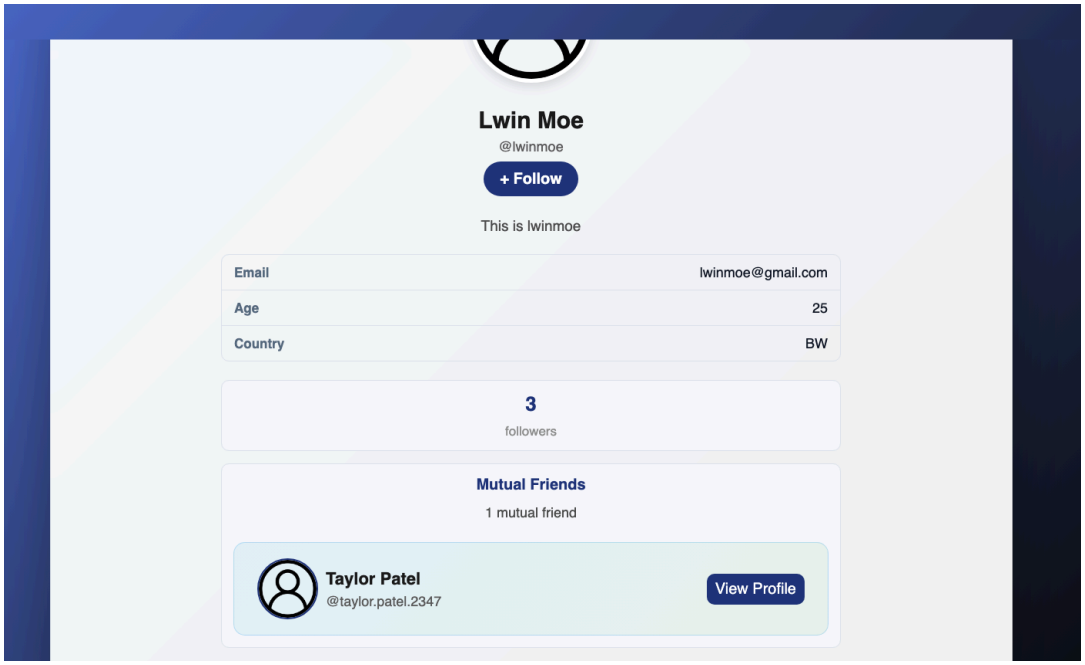
m.name AS name

LIMIT 10;

- Finds followed by BOTH user A and user B



In this example, we are trying to see if there are any mutual connections between User 4044 (Chirag) and User 4039 (Lwin Moe). Query returns one result.

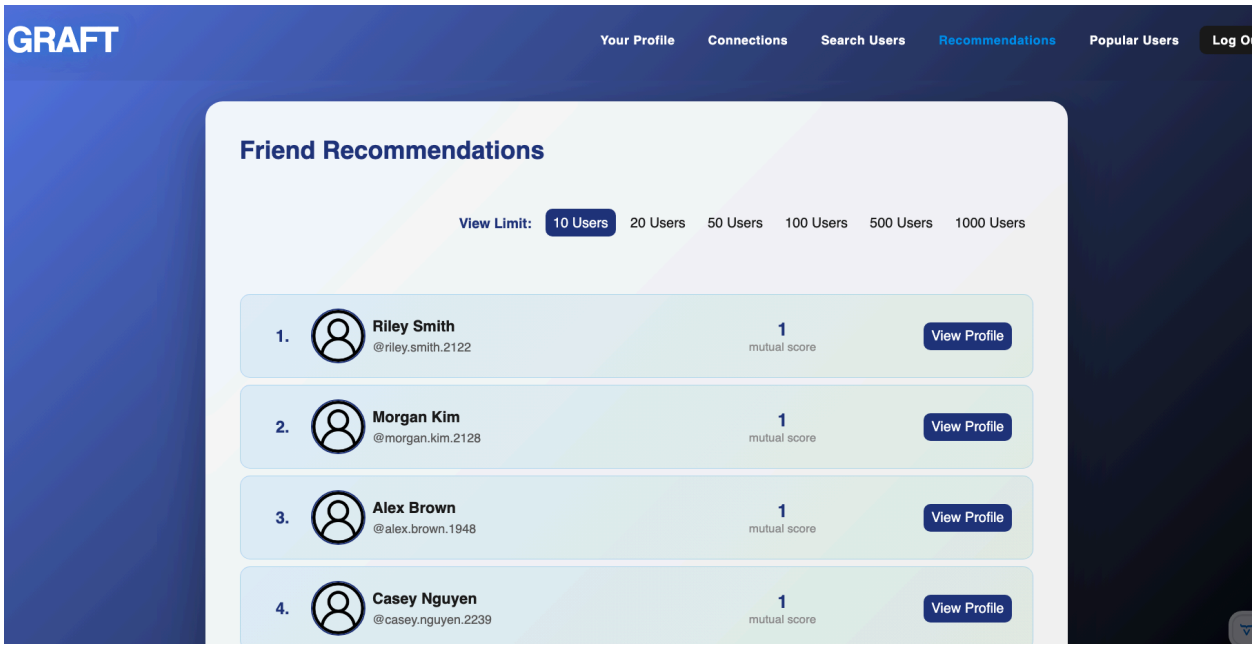


8) UC-9: Friend Recommendations

```
MATCH (u:User {user_id: "0"})-[:FOLLOWS]->(:User)-[:FOLLOWS]->(rec:User)
WHERE NOT (u)-[:FOLLOWS]->(rec) AND u <> rec
RETURN rec.user_id AS user_id,
    rec.username AS username,
    rec.name AS name,
    COUNT(*) AS mutual_connection_score
ORDER BY mutual_connection_score DESC
LIMIT 10;
```

The query looks 2 hops out, user '0' follows someone that follows 'rec'. 'Rec' is not already followed by user '0' and results are ranked by how many common paths lead to that recommendation.

Output: list of recommended users with a 'mutual_connection_score'



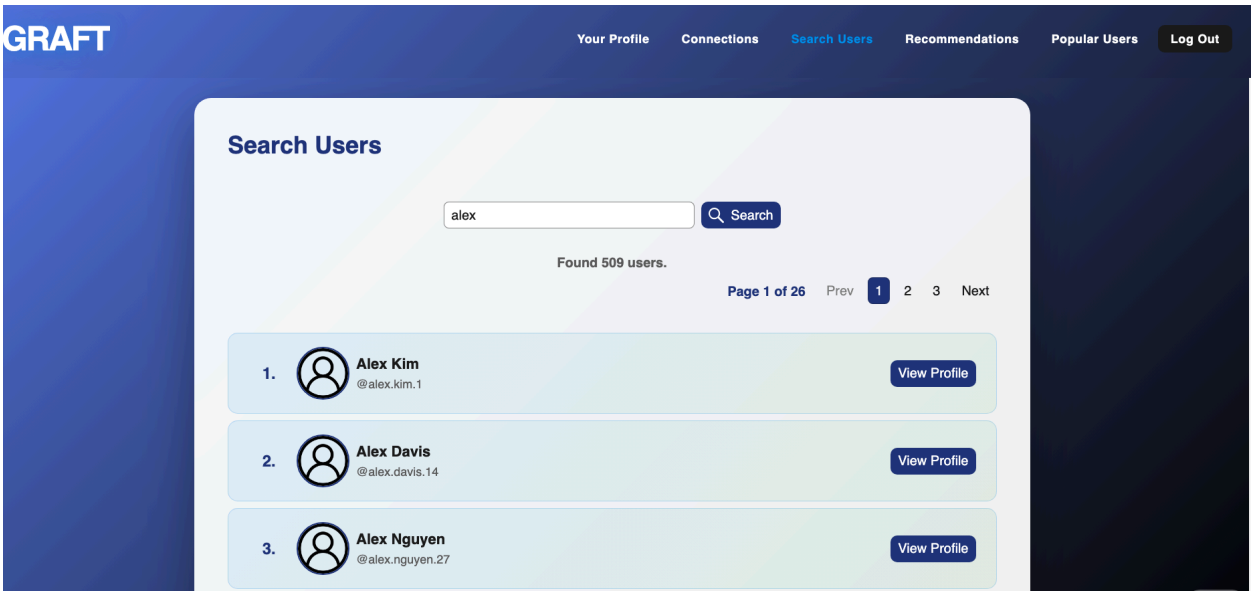
The query result and the display result match.

Search and Exploration

9) UC-10: Search Users

```
MATCH (u:User)
WHERE toLower(u.username) CONTAINS toLower("alex")
  OR toLower(u.name) CONTAINS toLower("alex")
RETURN u.user_id AS user_id,
    u.username AS username,
    u.name AS name,
    u.bio AS bio
LIMIT 10;
```

- Searches the ‘User’ nodes using partial matches on ‘username’ and ‘name’



Both the query and the result return 509 users containing "slex" in either their username or their name.

```
1 MATCH (u:User)
2 WHERE toLower(u.username) CONTAINS toLower("alex")
3    OR toLower(u.name) CONTAINS toLower("alex")
4 RETURN u.user_id AS user_id,
5        u.username AS username,
6        u.name AS name,
7        u.bio AS bio;
8
```

Table RAW

	user_id	username	name	bio
1	1	"alex.kim.1"	"Alex Kim"	"Alex Kim from IN. Activity score: 0.737."
2	14	"alex.davis.14"	"Alex Davis"	"Alex Davis from IN. Activity score: 0.766."
3	27	"alex.nguyen.27"	"Alex Nguyen"	"Alex Nguyen from SG. Activity score: 0.476."
4	44	"alex.brown.44"	"Alex Brown"	"Alex Brown from AU. Activity score: 0.602."
5	62	"alex.garcia.62"	"Alex Garcia"	"Alex Garcia from CA. Activity score: 0.205."

Started streaming 509 records after 58 ms and completed after 66 ms.

10) UC-11: Explore Popular Users

MATCH (u:User)<-[:FOLLOWS]-(f:User)

RETURN u.user_id AS user_id,

u.username AS username,

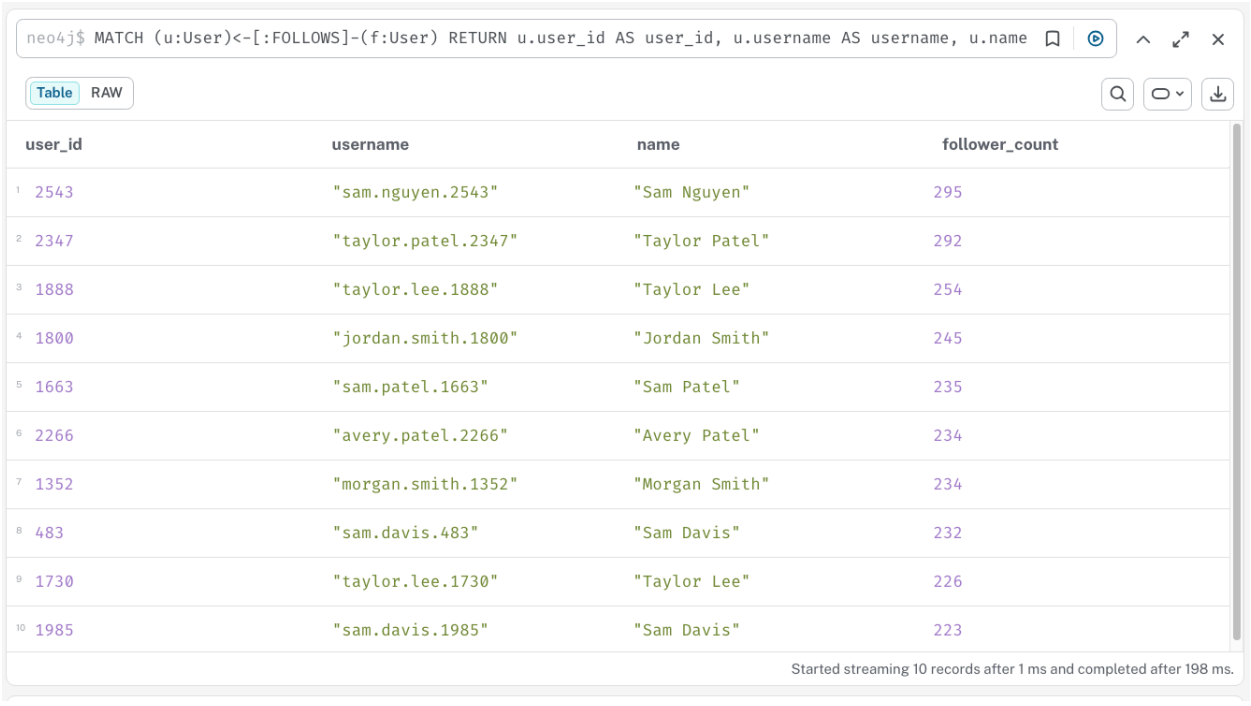
u.name AS name,

COUNT(f) AS follower_count

ORDER BY follower_count DESC

LIMIT 10;

- Counts incoming FOLLOWS relationships for each user and ranks them by follower count. This query outputs 10 most-followed users

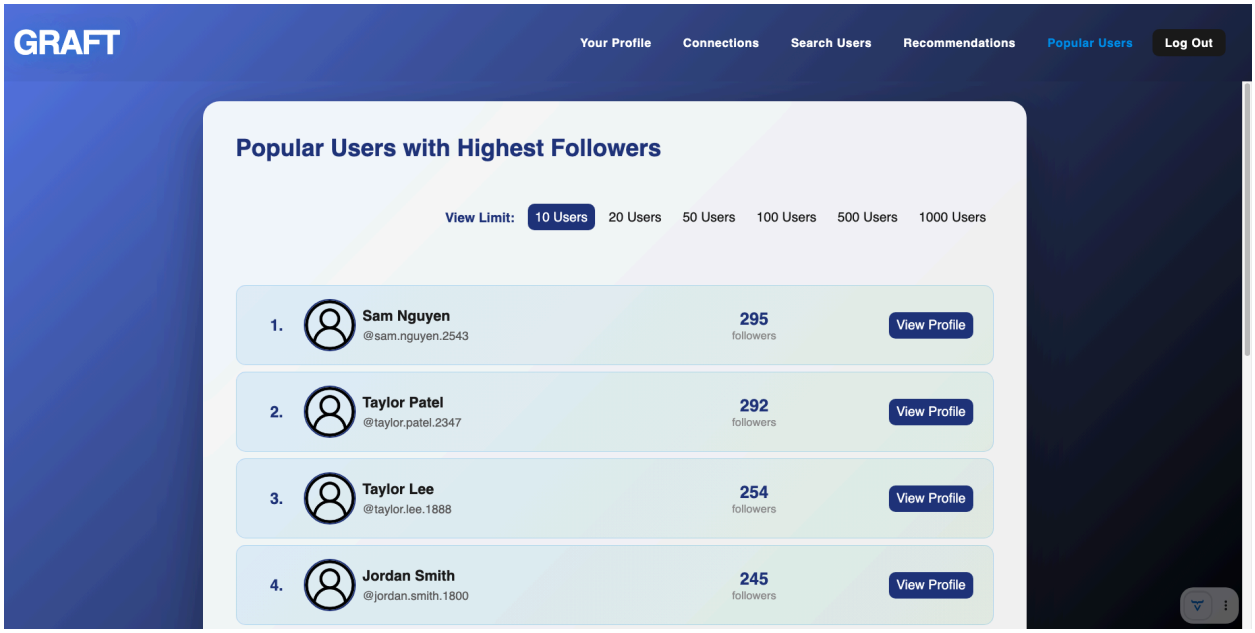


```
neo4j$ MATCH (u:User)<-[:FOLLOWS]-(f:User) RETURN u.user_id AS user_id, u.username AS username, u.name
```

Table RAW

	user_id	username	name	follower_count
1	2543	"sam.nguyen.2543"	"Sam Nguyen"	295
2	2347	"taylor.patel.2347"	"Taylor Patel"	292
3	1888	"taylor.lee.1888"	"Taylor Lee"	254
4	1800	"jordan.smith.1800"	"Jordan Smith"	245
5	1663	"sam.patel.1663"	"Sam Patel"	235
6	2266	"avery.patel.2266"	"Avery Patel"	234
7	1352	"morgan.smith.1352"	"Morgan Smith"	234
8	483	"sam.davis.483"	"Sam Davis"	232
9	1730	"taylor.lee.1730"	"Taylor Lee"	226
10	1985	"sam.davis.1985"	"Sam Davis"	223

Started streaming 10 records after 1 ms and completed after 198 ms.



Both the query result and the website returns the same result.